

Bài tập về Data Structures

Bài 1: String vs StringBuffer

Mô tả:

1. Viết chương trình thực hiện nối chuỗi văn bản.
2. **Hàm 1 (useString):** Tạo một chuỗi rỗng. Sử dụng vòng lặp `for` chạy 100.000 lần, mỗi lần cộng thêm chuỗi `"Hello"`. Đo thời gian chạy bằng `System.currentTimeMillis()`.
3. **Hàm 2 (useStringBuffer):** Sử dụng `StringBuffer` để làm việc tương tự (dùng hàm `append`). Đo thời gian chạy.
4. **Hàm 3 (contentAnalysis):** Cho một đoạn văn bản dài (có chứa ký tự đặc biệt, dấu câu...). Hãy dùng `String` hoặc `StringBuffer` để:
 - Đếm số lượng câu (dựa vào dấu chấm/chấm hỏi/chấm than).
 - Tìm và thay thế tất cả từ `"Java"` thành `"Python"` (Giả lập tính năng Find & Replace).

Bài 2: Customer support system

Mục tiêu: Áp dụng cơ chế FIFO (Queue) và LIFO (Stack)

Mô tả: Giả lập một hệ thống hỗ trợ khách hàng của Tiki/Shopee.

1. **Hàng đợi khách hàng (CustomerQueue - dùng LinkedList):**
 - Khách hàng gửi yêu cầu -> Thêm vào hàng đợi (`offer/add`).
 - Nhân viên xử lý -> Lấy khách hàng ra khỏi hàng đợi (`poll`).
 - Nếu hàng đợi rỗng mà vẫn bấm xử lý -> Thông báo "Không còn khách đợi".
2. **Lịch sử tin nhắn (MessageHistory - dùng Stack):**
 - Khi nhân viên gõ tin nhắn trả lời, lưu từng câu vào Stack.
 - Tính năng **Undo**: Nếu nhân viên gõ sai, bấm Undo sẽ xóa câu gần nhất vừa gõ (`pop`).
 - Tính năng **View Last**: Xem lại câu vừa gõ mà không xóa (`peek`).

Yêu cầu code:

- Tạo class `Customer(id, name)`, `Message(id, content)`, `Ticket(id, content, timestamp)` và các lớp khác nếu cần thiết.
- Viết hàm `main` giả lập quá trình: Khách A đến, Khách B đến, Xử lý A (gõ 3 dòng tin nhắn, undo 1 dòng), Xử lý B.

Bài 3: Word frequency counter

Mục tiêu: Kết hợp `String` xử lý, `Map` để đếm và `List` để sắp xếp.

Mô tả: Cho một đoạn văn bản tiếng Anh rất dài (khoảng 1 trang A4).

1. **Chuẩn hóa:** Chuyển về chữ thường, bỏ dấu câu (phẩy, chấm, ngoặc kép...).
2. **Đếm từ:** Sử dụng `HashMap<String, Integer>` để đếm số lần xuất hiện của từng từ.
 - Nếu từ chưa có trong Map -> put (word, 1).
 - Nếu từ đã có -> put (word, value cũ + 1).
3. **Sắp xếp & Thống kê:**
 - Tìm từ xuất hiện nhiều nhất.
 - Liệt kê các từ chỉ xuất hiện đúng 1 lần (Unique words).

Bài 4: Word frequency analyzer

Mục tiêu: Hiểu cơ chế hoạt động của `HashMap` (Key-Value), xử lý xung đột dữ liệu và so sánh hiệu năng với `ArrayList`.

Mô tả bài toán: Bạn cần xây dựng một công cụ phân tích văn bản cho một bài báo tiếng Anh. Công cụ này cần đếm xem mỗi từ xuất hiện bao nhiêu lần để tìm ra "từ khóa" chính của bài báo.

Dữ liệu đầu vào: Một đoạn văn bản (`String`) dài: *"Hello world. This is a java program. Hello java, hello world."*

Yêu cầu thiết kế & Cài đặt:

1. **Phân tích:**
 - Trường hợp 1: Dùng `ArrayList` để lưu từ.
 - Trường hợp 2: Dùng `HashMap<String, Integer>`: Key là từ (String), Value là số lần xuất hiện (Integer) để lưu từ.
 - So sánh độ phức tạp khi có thêm một từ mới và phải duyệt xem từ đó có chưa trong hai trường hợp.

2. Cài đặt Class **WordCounter**:

- Phương thức **analyze(String text)**:
 - B1: Chuẩn hóa chuỗi (chuyển về chữ thường, bỏ dấu chấm/phẩy).
 - B2: Tách chuỗi thành mảng các từ.
 - B3: Duyệt mảng. Với mỗi từ:
 - Nếu từ đã có trong Map (**containsKey**): Lấy giá trị cũ + 1 rồi **put** lại.
 - Nếu từ chưa có: **put** từ đó vào Map với giá trị 1.
- Phương thức **displayResult()**: In ra danh sách các từ và số lượng.
- Tìm và in ra từ xuất hiện nhiều nhất.
-

Bài 5: Library management system

Mục tiêu: Sau khi hoàn thành bài tập này, sinh viên cần:

- Áp dụng được các cấu trúc dữ liệu: arraylist, hashmap, treemap
- Hiểu sự khác nhau giữa List và Map
- So sánh hiệu năng tìm kiếm giữa các cấu trúc dữ liệu
- Phân tích độ phức tạp thời gian (Big-O)
- Biết lựa chọn cấu trúc dữ liệu phù hợp với từng tình huống

Mô tả bài toán: Xây dựng một hệ thống quản lý thư viện đơn giản.

Mỗi cuốn sách gồm các thông tin:

- Id (String) – Mã sách (duy nhất)
- Title (String) – Tên sách
- Author (String) – Tác giả
- Year (int) – Năm xuất bản

Hệ thống cần hỗ trợ các chức năng:

- Thêm sách
- Tìm sách theo id
- Xóa sách theo id
- In danh sách sách

Sinh viên cần cài đặt hệ thống bằng 3 cấu trúc dữ liệu khác nhau để so sánh hiệu năng và đặc điểm của từng cách lưu trữ.

Yêu cầu thiết kế & cài đặt

- 1) Thiết kế lớp dữ liệu
 - Tạo lớp Book chứa đầy đủ các thuộc tính cần thiết và constructor phù hợp.
- 2) Cài đặt hệ thống quản lý
 - Triển khai chức năng quản lý thư viện bằng: arraylist, hashmap, treemap
 - Mỗi cách lưu trữ phải đảm bảo thực hiện được các chức năng:
 - Thêm sách

- Tìm kiếm sách theo id
- Xóa sách theo id
- In danh sách sách

3) So sánh và phân tích

Sinh viên cần trả lời:

- Độ phức tạp khi tìm kiếm trong arraylist, hashmap và treemap.
- Cấu trúc dữ liệu nào phù hợp nhất khi:
 - Số lượng sách nhỏ
 - Số lượng sách rất lớn
 - Cần dữ liệu được sắp xếp theo id
- Vì sao hashmap thường tìm kiếm nhanh hơn arraylist?

Yêu cầu chương trình main

Trong hàm main:

- Thêm ít nhất 5 cuốn sách
- Thực hiện tìm kiếm
- Thực hiện xóa
- In danh sách kết quả
- Thực hiện với cả 3 cách lưu trữ

Bài tập về Xử lý ngoại lệ và Luồng vào ra

Bài 1: Phép chia an toàn

Viết chương trình đọc hai số nguyên từ bàn phím và thực hiện phép chia a / b . Kết quả trả về là một số nguyên.

Yêu cầu:

1. Dùng `Scanner` để nhập
2. Xử lý các ngoại lệ sau:
 - `InputMismatchException` khi người dùng nhập không phải số nguyên.
 - `ArithmeticException` khi chia cho 0.
3. Dù có lỗi hay không, chương trình phải in dòng: `Program finished.` bằng khối `finally`.
4. Nếu hợp lệ, in kết quả.

Bài 2: Sao chép tệp văn bản

Viết chương trình sao chép nội dung từ tệp nguồn sang tệp đích theo từng dòng.

Yêu cầu:

1. Nhập đường dẫn tệp nguồn và tệp đích từ bàn phím.
2. Dùng `FileReader` + `BufferedReader` để đọc và `FileWriter` + `PrintWriter` để ghi.
3. Bắt và xử lý các lỗi:
 - `FileNotFoundException`: báo `Source file not found.` hoặc `Cannot create destination file.`
 - `IOException`: báo `I/O error.` và in stack trace.
4. Đảm bảo đóng tệp trong `finally`.
5. In số dòng đã sao chép nếu thành công.

Bài 3: Đọc và ghi dữ liệu

Xây dựng hai chương trình để đọc và ghi dữ liệu.

Yêu cầu:

1. Chương trình ghi dữ liệu:

- Nhận `n` số nguyên từ bàn phím.
- Ghi tuần tự `n` số vào **tệp nhị phân** bằng `DataOutputStream` (ví dụ đặt tên file: `numbers.dat` hoặc `numbers.bin`).

2. Chương trình đọc dữ liệu:

- Đọc lại toàn bộ các số từ tệp đã ghi và in ra màn hình.
- Khi hết dữ liệu, dừng đọc bằng cách bắt `EOFException`.

3. Lưu ý:

- Tên tệp đọc/ghi nhập từ bàn phím.
- Xử lý `IOException` đầy đủ.

Bài 4: Danh sách sinh viên

Viết chương trình có thể nhập xuất danh sách các sinh viên.

Yêu cầu:

1. Xây dựng lớp `Student` gồm:

- `id` (String), `name` (String), `gpa` (double).
- Cài đặt `Serializable`.

2. Nhập danh sách sinh viên từ bàn phím cho tới khi gặp ký tự `"END"`.

3. Ghi danh sách ra tệp bằng `ObjectOutputStream`.

4. Đọc lại từ tệp bằng `ObjectInputStream` và in danh sách.

5. Bắt các ngoại lệ sau: `EOFException`, `ClassNotFoundException`, `FileNotFoundException`, `IOException`.

Bài 5: Đọc cấu hình + kiểm tra dữ liệu (Custom Exception)

Viết chương trình đọc **tệp cấu hình dạng text**, mỗi dòng có dạng `key=value`, sau đó kiểm tra tính hợp lệ của các tham số cấu hình.

Yêu cầu:

Nhập **đường dẫn file config** từ bàn phím bằng **Scanner**.

1. Đọc file theo từng dòng bằng **FileReader + BufferedReader**.
2. Mỗi dòng hợp lệ có dạng **key=value**:
 - Tách theo dấu **=**, lấy **key** và **value**.
 - Lưu vào **Map<String, String>**.
3. Kiểm tra dữ liệu:
 - Bắt buộc có **username** và **timeout**.
 - **timeout** phải là số nguyên và **> 0**.
 - Nếu có **maxConnections** thì phải là số nguyên và **>= 1**.
4. Xử lý ngoại lệ:
 - **FileNotFoundException**: in **Config file not found**.
 - **IOException**: in **I/O error**. và **in stack trace**
 - **NumberFormatException**: in **Invalid number format**.
 - Ngoại lệ tự định nghĩa **InvalidConfigException**: in **Invalid config: <message>**
5. Đảm bảo đóng file trong **finally**.
6. Dù lỗi hay không, chương trình **luôn** in: **Program finished**.
Nếu cấu hình hợp lệ, in toàn bộ cấu hình đọc được và in: **Config loaded successfully**.